

## Actividad Autónoma N.º 07

# Documentación de endpoints, DTO y pruebas de conexión

---

|                              |                                                                                                           |
|------------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>Estudiante:</b>           | Jhonsons Plasencia Valdez                                                                                 |
| <b>Asignatura:</b>           | Desarrollo de Aplicaciones Móviles                                                                        |
| <b>Unidad / Sesión:</b>      | 2 · Sesión 7 (Unidad 2, sesión 1)                                                                         |
| <b>Fecha de elaboración:</b> | 1 de octubre de 2026                                                                                      |
| <b>Proyecto:</b>             | PharmaMobile (Kotlin Multiplatform) sobre PharmaSoft                                                      |
| <b>Rama documentada:</b>     | feature/crud-productos-plasencia                                                                          |
| <b>Commit:</b>               | 661ba24                                                                                                   |
| <b>Repositorio:</b>          | <a href="https://github.com/Plasencia2007/pharmamobile">https://github.com/Plasencia2007/pharmamobile</a> |

Nota de alcance: esta documentación se elaboró junto con la implementación del CRUD completo de la Sesión 8 (que requiere, como insumo, el catálogo de endpoints y el diccionario de DTO aquí registrados). Por eso refleja el contrato final de las cinco operaciones sobre el recurso *productos*, no solo el GET de la sesión práctica 7.

## Producto 1 — Catálogo de endpoints

Recurso principal: **productos**. Backend: PharmaSoft (<https://github.com/dreyna/pharmaSoft>), Spring Boot + PostgreSQL. URL base: <http://10.0.2.2:8082/api/v1/> (Android, emulador) / <http://localhost:8082/api/v1/> (iOS, simulador). Versión de API: **v1**.

| Método | Ruta            | Parámetros                     | Respuesta esperada                           | Errores       |
|--------|-----------------|--------------------------------|----------------------------------------------|---------------|
| GET    | /productos      | pagina, tamaño (query)         | 200 · PaginaResponseDto<ProductoResponseDto> | 400           |
| GET    | /productos/{id} | id (ruta)                      | 200 · ProductoResponseDto                    | 400, 404      |
| POST   | /productos      | cuerpo ProductoRequestDto      | 201 · ProductoResponseDto                    | 400, 409      |
| PUT    | /productos/{id} | id + cuerpo ProductoRequestDto | 200 · ProductoResponseDto                    | 400, 404, 409 |
| DELETE | /productos/{id} | id (ruta)                      | 204 · sin cuerpo                             | 404, 409      |

**Nota sobre DELETE:** PharmaSoft implementa un borrado lógico (pone estado=false, no elimina la fila). Repetir un DELETE sobre un producto ya inactivo responde **409 Conflict** ("ya se encuentra inactivo"), no 204. Esto se descubrió durante la bitácora de pruebas (ver Producto 3, prueba adicional) y se documenta aquí porque no es evidente desde la guía genérica de la práctica.

## Producto 2 — Diccionario de DTO

**ProductoRequestDto (cuerpo enviado en POST/PUT):**

| Campo JSON  | Tipo Kotlin | Obligatorio | Valor por defecto | Campo en el dominio                   |
|-------------|-------------|-------------|-------------------|---------------------------------------|
| nombre      | String      | Sí          | —                 | Producto.nombre                       |
| precio      | Double      | Sí          | —                 | Producto.precio                       |
| stock       | Int         | Sí          | —                 | Producto.stock                        |
| estado      | Boolean     | No          | true              | Producto.activo                       |
| categoriald | Long        | Sí          | —                 | no mapeado en el dominio (solo envío) |

**ProductoResponseDto (lo que llega en cada GET):**

| Campo JSON      | Tipo Kotlin | Obligatorio | Valor por defecto | Campo en el dominio      |
|-----------------|-------------|-------------|-------------------|--------------------------|
| id              | Long        | Sí          | —                 | Producto.id              |
| nombre          | String      | Sí          | —                 | Producto.nombre          |
| precio          | Double      | Sí          | —                 | Producto.precio          |
| stock           | Int         | Sí          | —                 | Producto.stock           |
| estado          | Boolean     | No          | true              | Producto.activo          |
| categoriald     | Long?       | No          | null              | no mapeado en el dominio |
| categoriaNombre | String?     | No          | null              | no mapeado en el dominio |

**Fragmento real de la respuesta del servidor (GET /productos, un elemento):**

```
{
  "id": 4,
  "nombre": "Naproxeno 250mg",
```

```
"precio": 12.0,
"stock": 80,
"estado": true,
"categoriaId": 1,
"categoriaNombre": "Analgesicos",
"fechaCreacion": "2026-09-30T14:55:06.605996",
"fechaModificacion": null
}
```

Nótese que `fechaCreacion` y `fechaModificacion` llegan en toda respuesta real pero NO están declarados en `ProductoResponseDto`: la app no se rompe porque `ignoreUnknownKeys = true` en `HttpClientFactory.kt` los ignora. Ver Prueba 5 (Producto 3) para la evidencia de qué pasa si se desactiva esa opción.

### Código del DTO (data/remote/dto/ProductoDto.kt):

```
@Serializable
data class ProductoRequestDto(
    val nombre: String,
    val precio: Double,
    val stock: Int,
    val estado: Boolean = true,
    val categoriaId: Long
)

@Serializable
data class ProductoResponseDto(
    val id: Long,
    val nombre: String,
    val precio: Double,
    val stock: Int,
    val estado: Boolean = true,
    val categoriaId: Long? = null,
    val categoriaNombre: String? = null
)
```

## Producto 3 — Bitácora de pruebas de conexión

Las cinco pruebas se ejecutaron el 1 de octubre de 2026, entre las 19:05 y las 19:14 (hora UTC del log de Ktor), en el emulador Android PharmaMobile\_Test (Pixel, API 35), contra el backend PharmaSoft corriendo en localhost:8082.

### Prueba 1 · Respuesta exitosa

|                     |                                                                                                |
|---------------------|------------------------------------------------------------------------------------------------|
| Fecha y hora        | 01/10/2026, 19:05 UTC                                                                          |
| Plataforma          | Android (emulador Pixel, API 35)                                                               |
| Pasos               | Backend activo → abrir la app → pestaña Productos → se dispara GET /productos?pagina=0&tama    |
| Resultado esperado  | Código 200 y lista renderizada en la interfaz.                                                 |
| Resultado observado | 200 OK; 16 productos reales listados, repartidos en las pestañas Activos/Inactivos/Bajo stock. |

```
adb logcat -- pe.edu.upeu.pharmamobile (System.out) -- Prueba 1: listado exitoso

10-01 19:05:10.930 3825 3825 I System.out: HttpClient: REQUEST: http://10.0.2.2:8082/api/v1/productos?pagina=0&tamano=20
10-01 19:05:10.930 3825 3825 I System.out: METHOD: GET
10-01 19:05:10.930 3825 3825 I System.out: COMMON HEADERS
10-01 19:05:10.930 3825 3825 I System.out: -> Accept: application/json
10-01 19:05:10.930 3825 3825 I System.out: -> Content-Type: application/json
10-01 19:05:10.931 3825 3825 I System.out: CONTENT HEADERS
10-01 19:05:10.931 3825 3825 I System.out: -> Content-Length: 0
10-01 19:05:11.656 3825 3825 I System.out: HttpClient: RESPONSE: 200
10-01 19:05:11.657 3825 3825 I System.out: METHOD: GET
10-01 19:05:11.658 3825 3825 I System.out: FROM: http://10.0.2.2:8082/api/v1/productos?pagina=0&tamano=20
10-01 19:05:11.658 3825 3825 I System.out: COMMON HEADERS
10-01 19:05:11.658 3825 3825 I System.out: -> connection: keep-alive
10-01 19:05:11.658 3825 3825 I System.out: -> content-type: application/json
10-01 19:05:11.658 3825 3825 I System.out: -> date: Thu, 01 Oct 2026 19:05:12 GMT
10-01 19:05:11.659 3825 3825 I System.out: -> keep-alive: timeout=60
10-01 19:05:11.659 3825 3825 I System.out: -> transfer-encoding: chunked
10-01 19:05:11.659 3825 3825 I System.out: -> vary: Origin; Access-Control-Request-Method; Access-Control-Request-Headers
```

Figura 1. Log de Ktor: REQUEST GET → RESPONSE 200.

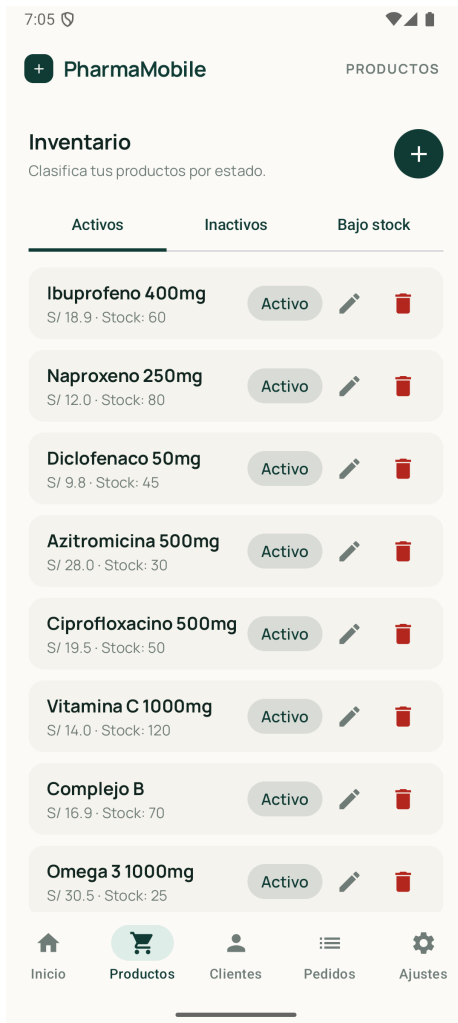


Figura 2. Lista de productos renderizada tras la respuesta 200.

**Conclusión: el GET paginado funciona end-to-end contra el backend real, sin errores.**

Prueba 2 · Recurso inexistente

|                     |                                                                                                      |
|---------------------|------------------------------------------------------------------------------------------------------|
| Fecha y hora        | 01/10/2026, 19:06–19:08 UTC                                                                          |
| Plataforma          | Android (emulador Pixel, API 35)                                                                     |
| Pasos               | 1) Con la lista cargada, eliminar el producto id=2 directamente por API (curl -X DELETE .../producto |
| Resultado esperado  | ClientRequestException con código 404 y mensaje controlado.                                          |
| Resultado observado | PUT /productos/2 → 404; la app muestra "El producto ya no existe." (ErrorApi.NoEncontrado) sin cr    |

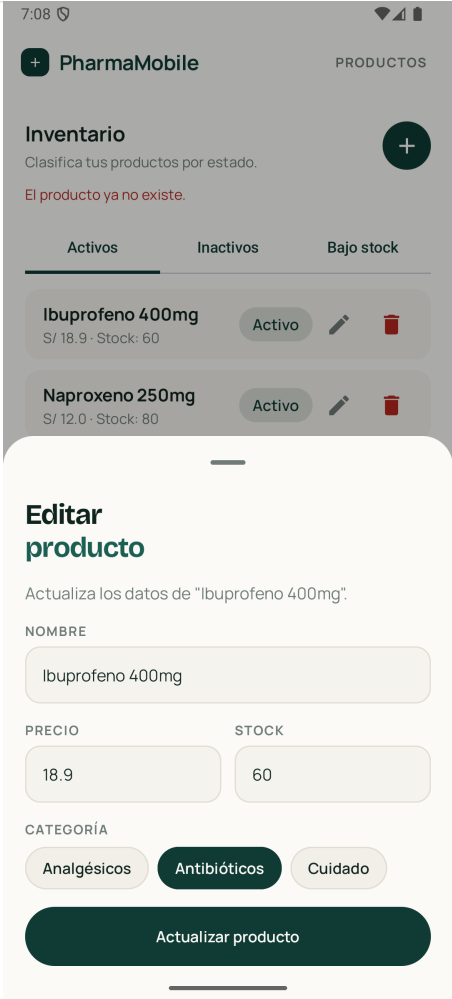


Figura 3. "El producto ya no existe." tras un PUT contra un id eliminado (404).

Hallazgo adicional (no pedido por la guía, mantenido porque documenta el contrato real): un DELETE repetido sobre un producto ya inactivo responde **409 Conflict** ("El producto Ibuprofeno 400mg ya se encuentra inactivo"), porque PharmaSoft hace borrado lógico. Ese 409 también se maneja sin crash, vía ErrorApi.Conflicto.

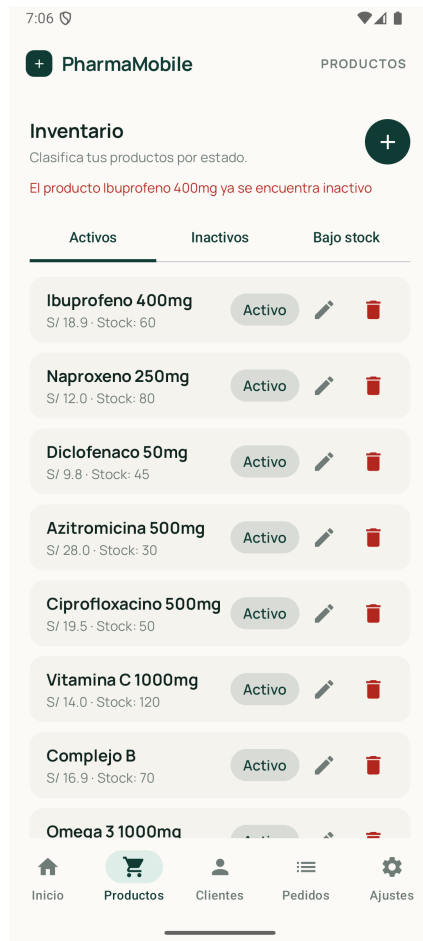


Figura 4 (bonus). 409 Conflict al reintentar DELETE sobre un producto ya inactivo.

**Conclusión: los errores 404 y 409 del servidor llegan a la UI como mensajes legibles, sin crash.**

Prueba 3 · Sin conexión

|                     |                                                                                                        |
|---------------------|--------------------------------------------------------------------------------------------------------|
| Fecha y hora        | 01/10/2026, 19:09 UTC                                                                                  |
| Plataforma          | Android (emulador Pixel, API 35)                                                                       |
| Pasos               | adb shell svc wifi disable && svc data disable → forzar relanzar la app → pestaña Productos (disparar) |
| Resultado esperado  | Excepción de entrada/salida capturada y mensaje al usuario.                                            |
| Resultado observado | "No hay conexión con el servidor." (ErrorApi.SinConexion, vía kotlin.io.IOException capturada en e     |

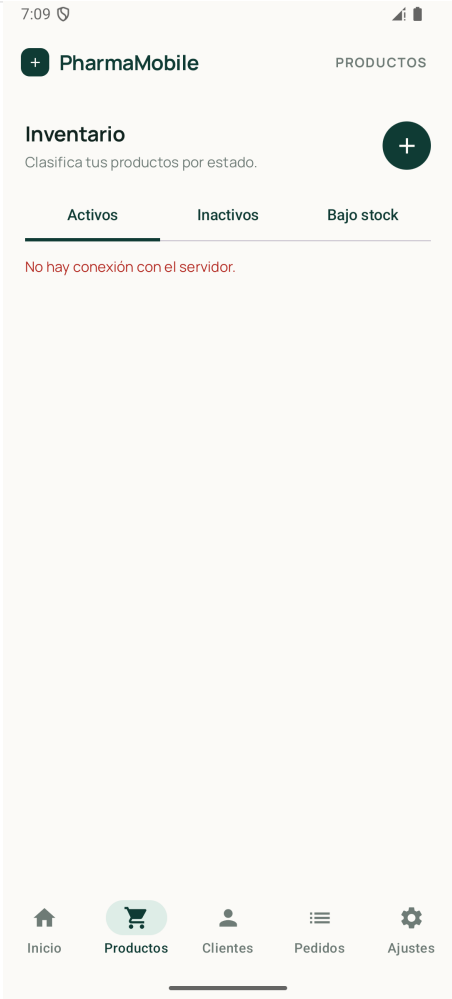


Figura 5. "No hay conexión con el servidor." con WiFi y datos desactivados.

**Conclusión: la falta de conectividad se traduce a un mensaje claro, sin excepción no controlada.**

Prueba 4 · Tiempo de espera agotado

|                     |                                                                                                     |
|---------------------|-----------------------------------------------------------------------------------------------------|
| Fecha y hora        | 01/10/2026, 19:12–19:13 UTC                                                                         |
| Plataforma          | Android (emulador Pixel, API 35)                                                                    |
| Pasos               | Editar temporalmente HttpClientFactory.kt: requestTimeoutMillis = 1 → recompilar e instalar → relar |
| Resultado esperado  | HttpRequestTimeoutException controlada; la interfaz no se congela.                                  |
| Resultado observado | "La solicitud tardó demasiado. Intenta nuevamente." (ErrorApi.TiempoAgotado); la pantalla responde  |

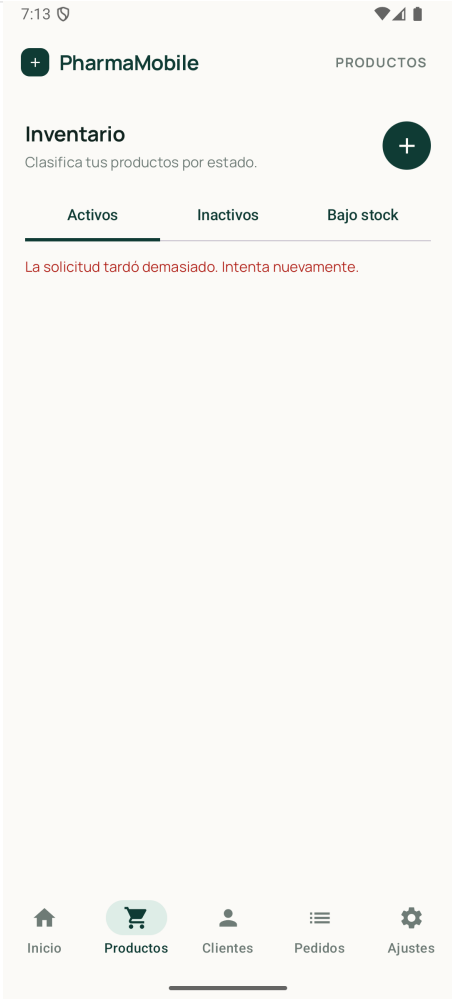


Figura 6. Mensaje de timeout con requestTimeoutMillis = 1.

**Conclusión:** el timeout configurado evita que una petición lenta congele la UI; el cambio se revirtió tras la prueba.

Prueba 5 · Campo desconocido en el JSON

|                     |                                                                                                  |
|---------------------|--------------------------------------------------------------------------------------------------|
| Fecha y hora        | 01/10/2026, 19:14 UTC                                                                            |
| Plataforma          | Android (emulador Pixel, API 35)                                                                 |
| Pasos               | Editar temporalmente HttpClientFactory.kt: ignoreUnknownKeys = false → recompilar e instalar → r |
| Resultado esperado  | Con la opción activa la aplicación continúa; sin ella se produce SerializationException.         |
| Resultado observado | La app CRASHEA (vuelve al launcher) con kotlinx.serialization.json.JsonDecodingException: "Encor |

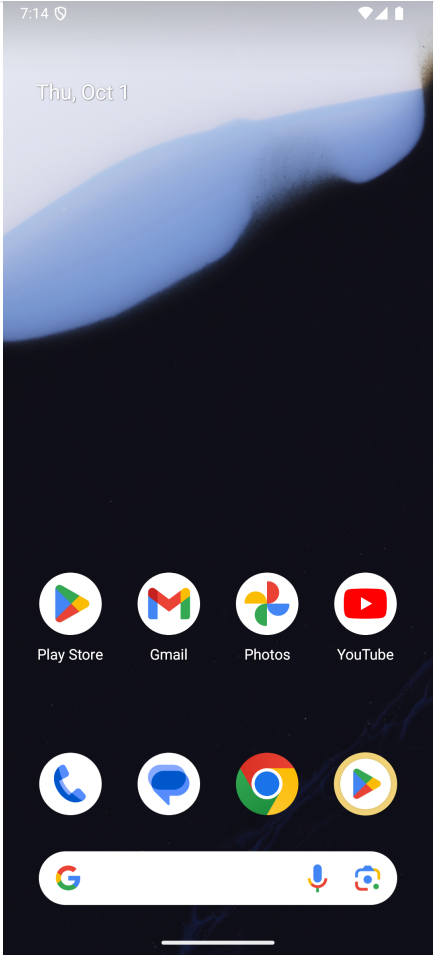


Figura 7. La app vuelve al launcher (crash) tras el JsonDecodingException.

Fragmento del stack trace (docs/evidencias/prueba5\_stacktrace.txt):

```
Caused by: kotlinx.serialization.json.JsonDecodingException:
Unexpected JSON token at offset 142: Encountered an unknown key
'fechaCreacion' at path: $.contenido[0]
Use 'ignoreUnknownKeys = true' in 'Json {}' builder or
'@JsonIgnoreUnknownKeys' annotation to ignore unknown keys.
    at ...StreamingJsonDecoder.handleUnknown(StreamingJsonDecoder.kt:258)
    at ...ProductoResponseDto$Serializer.deserialize(ProductoDto.kt:21)
    at pe.edu.upeu.pharmamobile.data.remote.ProductoApi.listar(ProductoApi.kt:47)
```

Conclusión: **ignoreUnknownKeys = true** no es opcional en este proyecto — sin él, cualquier campo nuevo que PharmaSoft agregue a futuro (p. ej. auditoría) tumba la app en producción. Esta prueba confirma por qué esa configuración debe permanecer activa, y expone una mejora pendiente: ejecutarLlamada() podría además capturar **SerializationException** para mostrar un mensaje controlado en vez de crashear, por si algún día se desactiva por error. Tras la prueba se revirtió `ignoreUnknownKeys = true`, se recompiló y se confirmó que el listado vuelve a cargar con normalidad (ver Figura 2).

## Producto 4 — Evidencias y repositorio

| Evidencia                                   | Ubicación                                                                                                    |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Log de Ktor (petición + respuesta completa) | docs/evidencias/ktor_log_prueba1.png / .txt                                                                  |
| App corriendo en Android (emulador)         | docs/evidencias/prueba1_exitosa.png                                                                          |
| App corriendo en iOS (simulador)            | No disponible: no se cuenta con macOS/Xcode en esta máquina de desarrollo. Validado solo por el profesor     |
| Repositorio                                 | <a href="https://github.com/Plasencia2007/pharmamobile">https://github.com/Plasencia2007/pharmamobile</a>    |
| Rama / commit documentado                   | feature/crud-productos-plasencia @ 661ba24                                                                   |
| README actualizado                          | Sección "Conectividad REST" en README.md (catálogo de endpoints, diccionario de DTO, mapeo de DTO a entidad) |

## Lista de cotejo previa a la entrega

| N.º | Criterio observable                                                | Cumple                                         |
|-----|--------------------------------------------------------------------|------------------------------------------------|
| 1   | PDF nombrado S07_ActividadAutonoma_Plasencia.pdf                   | Sí                                             |
| 2   | Portada con nombre completo, sesión, fecha y enlace al repositorio | Sí                                             |
| 3   | Catálogo con al menos 5 endpoints con todos sus datos              | Sí                                             |
| 4   | Diccionario documenta todos los campos de cada DTO                 | Sí                                             |
| 5   | Fragmento de JSON real junto al código del DTO                     | Sí                                             |
| 6   | Bitácora con los 5 escenarios y su captura respectiva              | Sí                                             |
| 7   | Cada escenario de error indica qué ve el usuario final             | Sí                                             |
| 8   | Capturas de ejecución en Android y en iOS                          | Parcial (solo Android; iOS sin Mac disponible) |
| 9   | README con la sección de conectividad REST actualizada             | Sí                                             |
| 10  | El commit indicado existe y es accesible                           | Sí                                             |